

MicroBlend

System Feature List

Complete feature documentation for the
MicroBlend Point-of-Sale & Table Management System

Generated from the Django 6.0.3 + DRF codebase

MicroBlend — System Feature List

Overview

MicroBlend is a Django 6.0.3 + DRF backend for a spatially distributed restaurant operation. The system manages the full lifecycle of a dine-in experience: customer QR scan-in, order placement, kitchen/bar preparation workflows, cashier checkout, inventory deduction, and external POS synchronisation — all connected through server-sent events for real-time updates.

1. User Management & Authentication

Core identity and access control for all actor types: customers, waiters, kitchen/bar staff, cashiers, and administrators.

Endpoints

Method	Route	Description
POST	<code>`/api/identity/register/`</code>	Register a new customer account (email or phone + password)
POST	<code>`/api/identity/login/`</code>	Authenticate by email/phone + password; returns token
POST	<code>`/api/identity/logout/`</code>	Invalidate current auth token
GET	<code>`/api/identity/me/`</code>	Retrieve or update own profile
PATCH	<code>`/api/identity/me/`</code>	Update profile fields (name, email, phone, password)
DELETE	<code>`/api/identity/me/`</code>	Soft-delete own account
GET	<code>`/api/identity/me/history/`</code>	View own action log + linked guest orders
POST	<code>`/api/identity/guest/start/`</code>	Create a temporary guest session (walk-in, no registration)
GET	<code>`/api/identity/guest/session/`</code>	Check guest session validity and remaining time
GET/PATCH	<code>`/api/admin/users/`</code>	Admin: list, retrieve, update, or deactivate any user

Key Behaviors

- **Guest access:** Walk-in customers can order without registration; guest sessions expire after 12 hours (configurable via `GUEST\_ACCESS\_HOURS`). Guest orders are linked to the permanent account when the customer later registers with the same device.
- **Device binding:** On first login, the user's device ID is bound to the account; subsequent logins from other devices are rejected.
- **Soft delete:** Account deletion sets `is\_deleted = True` and `is\_active = False`; rows are never physically removed, preserving foreign key integrity.
- **Staff sub-roles:** Staff accounts carry a `staff\_role` subdivision (waiter, kitchen, bar, cashier) that gates access to specific endpoints.
- **Audit trail:** Every auth event (register, login, logout, delete) is logged to `AuditLog`.

2. Menu & Categories

Manages the sellable item catalogue, grouped by category.

Endpoints

Method	Route	Description
GET/POST	`/api/categories/`	List or create menu categories (staff/admin only for write)
GET/PUT/PATCH/DELETE	`/api/categories/{id}/`	Retrieve, update, or delete a category
GET/POST	`/api/menu-items/`	List or create menu items
GET/PUT/PATCH/DELETE	`/api/menu-items/{id}/`	Retrieve, update, or delete a menu item
GET/POST	`/api/order-playlists/`	List or create saved order playlists (authenticated users only)
GET/PUT/PATCH/DELETE	`/api/order-playlists/{id}/`	Retrieve, update, or delete a playlist

Key Behaviors

- **Availability auto-toggle:** `MenuItem.recalculate\_availability()` checks whether all ingredient requirements can be met from current stock; items are automatically marked unavailable when ingredient stock runs out.
- **Popularity tracking:** Each item has a `popularity\_score` that increments when ordered; the menu can be sorted by popularity for customer display.
- **Preparation station routing:** Each item is tagged `kitchen` or `bar`; this determines which station's workflow processes the item.
- **Customer-facing filter:** The `?audience=customer` query parameter returns only available items with a simplified serializer.
- **Order playlists:** Authenticated users can create named playlists (reusable order templates) for repeat ordering.

3. Inventory Management

Tracks ingredient stock levels, batch deliveries, consumption, and low-stock alerts.

Endpoints

Method	Route	Description
GET/POST	`/api/inventory/ingredients/`	List or create ingredients
GET/PUT/PATCH/DELETE	`/api/inventory/ingredients/{id}/`	Retrieve, update, or delete an ingredient
GET/POST	`/api/inventory/batches/`	List or create inventory batches (restock)
GET/PUT/PATCH/DELETE	`/api/inventory/batches/{id}/`	Retrieve, update, or delete a batch
GET	`/api/inventory/movements/`	List all inventory movements (admin only)
GET	`/api/inventory/movements/{id}/`	Retrieve a specific movement

Key Behaviors

- **FIFO consumption:** When ingredients are consumed (during order preparation), stock is deducted from the oldest batch first (`expiration\_date ASC`), minimising waste from spoilage.
- **Batch restocking:** Each delivery is recorded as an `InventoryBatch` with quantity added, unit cost, and expiration date; the running `quantity\_remaining` is decremented as stock is consumed.
- **Immutable movement log:** Every stock change (restock, deduct, adjust, delete) creates an `InventoryMovement` audit record that references the actor, ingredient, batch, and optionally the related order.
- **Low-stock detection:** When `available\_quantity <= reorder\_level`, the dashboard highlights the ingredient and real-time events are broadcast.
- **Menu availability cascade:** After any stock change, `recalculate\_menu\_availability()` re-checks all menu items and broadcasts availability changes via SSE.
- **Stock projection:** The `ingredient\_stock\_projection()` service returns a snapshot of current stock levels vs. reorder thresholds for planning.

4. Order Management

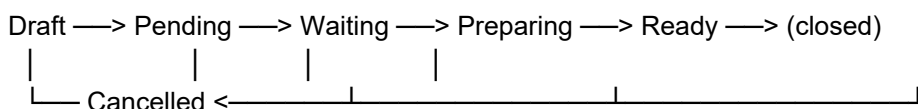
The central feature — handles the full order lifecycle from draft creation through cashier checkout.

Endpoints

Method	Route	Description
GET/POST	`/api/orders/`	List or create orders
GET/PUT/PATCH/DELETE	`/api/orders/{id}/`	Retrieve, update, or delete an order (patch updates draft items)
POST	`/api/orders/{id}/submit/`	Submit order for processing
POST	`/api/orders/{id}/cancel/`	Cancel an order
POST	`/api/orders/{id}/kitchen_update/`	Kitchen staff: mark station status (preparing/ready)
POST	`/api/orders/{id}/bar_update/`	Bar staff: mark station status (preparing/ready)
POST	`/api/orders/{id}/cashier_update/`	Cashier: confirm, mark paid/unpaid
POST	`/api/orders/from_playlist/`	Create an order from a saved playlist
GET	`/api/order-items/`	List all order items (staff/admin only)
GET	`/api/order-items/{id}/`	Retrieve a specific order item

Order Lifecycle

...



...

- **Draft:** Order is being built; items can be freely added/removed/modified. Created by customer or waiter.
- **Pending:** Submitted for cashier confirmation (if high-value, a threshold alert fires at PHP 2,500).
- **Waiting:** Cashier has confirmed the order; kitchen/bar can now see it.

- **Preparing:** At least one station is actively working on items. Inventory is deducted at this point.
- **Ready:** All required stations have finished preparation.
- **Cancelled:** Order was voided before or during preparation.

Key Behaviors

- **Parallel station tracks:** ``kitchen_status`` and ``bar_status`` advance independently via ``kitchen_update`` / ``bar_update``. The overall order status is derived from both — ``Preparing`` if any station is preparing, ``Ready`` only when both required stations are done.
- **Cashier actions:** Cashier can move ``Pending`` → ``Waiting`` (confirm), mark orders as ``Paid``, or revert a paid order (requires credential re-verification with a reason).
- **Guest ordering:** Unauthenticated users place orders with a ``guest_key``; guest orders are linked to the device and later transferable to a registered account.
- **Idempotency guard:** ``enforce_debounce`` prevents duplicate order submissions within a configurable window (default 5 seconds), using ``DebounceRecord`` unique constraints.
- **Inventory deduction:** Dedicated to the station (kitchen/bar) at the start of preparation via ``_deduct_inventory_for_station()``, which aggregates ``MenuitemIngredient`` requirements across all order items belonging to that station.
- **Demand alerts:** When an item's recent order volume exceeds 5 or its ingredients are low, a role-targeted notification is sent to the relevant station.
- **Bulk orders:** Flagged ``is_bulk_order``; supports higher per-item quantities (over 20) and requires cashier verification.
- **Snapshot fields:** ``item_name``, ``station``, ``unit_price`` are copied at order time so historical receipts survive menu changes. ``placed_by_name`` and ``placed_by_role`` similarly survive user deletion.

5. Table Management

Physical table configuration, status tracking, merge groups, QR scan requests, and staff page calls.

Endpoints

Method	Route	Description
GET/POST	<code>`/api/tables/`</code>	List or create tables
GET/PUT/PATCH/DELETE	<code>`/api/tables/{id}/`</code>	Retrieve, update, or delete a table
POST	<code>`/api/tables/{id}/mark_occupied/`</code>	Mark table as occupied (staff)
POST	<code>`/api/tables/{id}/mark_vacant/`</code>	Mark table as vacant (staff)
GET/POST	<code>`/api/table-groups/`</code>	List or create merge groups
GET/PUT/PATCH/DELETE	<code>`/api/table-groups/{id}/`</code>	Retrieve, update, or delete a merge group
GET/POST	<code>`/api/table-scan-requests/`</code>	List or create QR scan requests
GET/PUT/PATCH/DELETE	<code>`/api/table-scan-requests/{id}/`</code>	Retrieve, update, or delete a scan request
POST	<code>`/api/table-scan-requests/{id}/moderate/`</code>	Cashier: approve or block a scan request
GET/POST	<code>`/api/staff-pages/`</code>	List or create staff page requests
GET/PUT/PATCH/DELETE	<code>`/api/staff-pages/{id}/`</code>	Retrieve, update, or delete a page request
POST	<code>`/api/staff-pages/{id}/finish/`</code>	Staff: mark a page request as finished

Key Behaviors

- **QR code table scanning:** Each table has an auto-generated ``qr_code_value``. Customers scan the QR to send a ``TableScanRequest``; cashiers moderate the request (approve/block). On approval, a ``TableSession`` is automatically created.
- **Table merging:** Multiple tables can be grouped via ``TableMergeGroup`` for large parties; combined capacity is computed from member tables.
- **Staff paging:** Customers can request staff assistance with specific reasons: table cleanup, payment, or accident. Staff mark requests as finished after resolution.

- **Debounced paging:** Staff page requests are debounced (configurable, default 10 seconds) to prevent spam.

6. Table Sessions

Seating sessions that track which customer is at which table and for how long.

Endpoints

GET/PUT/PATCH/DELETE

Method	Route	Description
GET/POST	<code>`/api/table-sessions/`</code>	List or create seating sessions (staff/admin)
GET/PUT/PATCH/DELETE	<code>`/api/table-sessions/{id}/`</code>	Retrieve, update, or delete a session
POST	<code>`/api/table-sessions/{id}/close/`</code>	Close a session (staff/admin)

Key Behaviors

- **Session sources:** Sessions can be created from QR scan approval, walk-in, waiter assignment, or manual entry.
- **Party tracking:** ``party_size`` and ``guest_label`` record customer information for the session duration.
- **Active state:** ``is_active`` tracks whether the session is currently open; closing a session ends it and typically triggers table vacancy.
- **Order association:** All orders placed during the session are linked via ``table_session_id``.

7. Notifications

In-app notification system supporting both user-targeted and role-targeted delivery.

Endpoints

Method	Route	Description
GET	<code>`/api/notifications/`</code>	List notifications for the current user (own + role-targeted)
PATCH	<code>`/api/notifications/{id}/`</code>	Mark notification as read

Key Behaviors

- **Role-targeted delivery:** Notifications can target a role (e.g., "cashier", "kitchen") without specifying individual users; all users matching that role see it.
- **User-targeted delivery:** Direct notifications to a specific user.
- **Debounce records:** The ``DebounceRecord`` model enforces rate-limiting for actions like order submission and staff paging. Unique constraint on ``(actor_key, action, object_key)`` prevents duplicate rapid-fire actions.

8. Real-Time Events (SSE)

Polling-based server-sent events stream for live UI updates across all devices.

Endpoints

Method	Route	Description
GET	<code>`/api/realtime/events/`</code>	Fetch events after a cursor (<code>`after_id`</code>)
GET	<code>`/api/realtime/stream/`</code>	SSE streaming endpoint; polls every 1 second, yields new events

Key Behaviors

- **Polling SSE:** The stream endpoint loops every 1 second, querying for events with `id > cursor`. Returns formatted SSE lines (`event:`, `data:`, `id:`). After `timeout` seconds (default 30), a `keepalive` event is sent and the connection closes.
- **Event targeting:** Events are delivered based on:
 - `role\_target` — any user whose role or staff_role matches
 - `user` — a specific recipient
 - Untargeted events (no role_target, no user) are broadcast to everyone
- **Guest visibility:** Guest users can only see untargeted events and events explicitly addressed to their guest user ID.
- **Event types include:** `order.draft\_saved`, `order.submitted`, `order.cancelled`, `order.kitchen\_status\_changed`, `order.bar\_status\_changed`, `order.cashier\_status\_changed`, `menu.availability\_changed`, `inventory.batch\_restocked`, `inventory.ingredient\_stock\_changed`, and more.

9. Integration & External Sync

Outbox-pattern event log for pushing data changes to external systems (POS, mobile, kiosk).

Endpoints

Method	Route	Description
GET/POST	`/api/integrations/external-systems/`	Admin: manage registered external systems
GET/PUT/PATCH/DELETE	`/api/integrations/external-systems/{id}/`	Admin: edit or remove an external system
GET	`/api/integrations/sync-events/`	List sync events (staff/admin)
GET	`/api/integrations/sync-events/{id}/`	Retrieve a sync event
GET	`/api/integrations/sync-events/latest_cursor`	Get the highest sync event ID
POST	`/api/integrations/sync-events/{id}/acknowledged`	Mark event as delivered or failed
POST	`/api/integrations/sync-events/retry_due/`	Admin: retry all due failed events

Key Behaviors

- **Outbox pattern:** Domain services call `publish\_sync\_event()` during the main request transaction; events are stored in `SyncEvent` and consumed asynchronously by an external worker.
- **Idempotency:** Each event carries an optional `idempotency\_key` (unique) to prevent duplicate processing by consumers.
- **Exponential backoff:** Failed events are retried with exponential backoff (`base \* 2^(attempt-1)` seconds), up to 8 attempts (configurable). After max retries, the event is dropped.
- **Delivery statuses:** Pending → Retry → Delivered / Failed / Dropped.

10. Analytics & Reporting

Business intelligence: dashboard snapshots, report generation, cost simulation, database backup and reset.

Endpoints

Method	Route	Description
GET	`/api/analytics/dashboard/`	Dashboard snapshot for a date range (daily/weekly/monthly/annual/custom)
POST	`/api/analytics/reports/generate/`	Generate and save a report
GET	`/api/analytics/reports/`	List saved reports
GET	`/api/analytics/reports/{id}/`	Retrieve a specific report
POST	`/api/analytics/simulate/`	Run a what-if cost simulation
GET	`/api/analytics/simulations/`	List simulations
GET	`/api/analytics/simulations/{id}/`	Retrieve a simulation
POST	`/api/analytics/backup/`	Admin: create a full SQL dump backup

POST	<code>`/api/analytics/reset/`</code>	Admin: backup then flush the database
------	--------------------------------------	---------------------------------------

Dashboard Metrics

- Order count and paid order count
- Total revenue
- Top 5 and bottom 5 menu items by quantity
- Table status distribution
- Open staff page requests
- Low-stock ingredient alerts

Key Behaviors

- **Cost simulation:** Users input assumptions (menu price delta, salary changes, staff count changes, expansion costs) and the system projects revenue, costs, and profit.
- **Daily digest:** ``send_daily_digest_to_admins()`` automatically generates a daily report snapshot and notifies all administrators.
- **Backup/Restore:** The backup action dumps the full SQLite database using ``connection.iterdump()``; the reset action performs a backup first then flushes all data.

11. Audit Logging

Polymorphic audit trail that records all significant user actions across the system.

Endpoints

(No public API — audit logs are created internally via ``log_user_action()`` and exposed through the user's own history endpoint ``/api/identity/me/history/``)

Key Behaviors

- **Polymorphic target reference:** Uses ``target_type`` (class name) + ``target_id`` (PK string) + ``target_label`` (string representation) to reference any model without formal foreign keys.
- **Action naming convention:** Actions follow a ``domain.verb`` pattern (e.g., ``order.submitted``, ``inventory.restocked``, ``account.logged_in``, ``admin.account.deactivated``).
- **Role snapshot:** The actor's role at the time of the action is captured in ``actor_role`` even if the role changes later.

12. Feedback System

Customer feedback and problem report submission with staff review workflow.

Endpoints

Method	Route	Description
GET/POST	<code>`/api/feedback/`</code>	List or create feedback entries (authenticated users)
GET/PUT/PATCH/DELETE	<code>`/api/feedback/{id}/`</code>	Retrieve, update, or delete (staff/admin can manage all)

Key Behaviors

- **Two entry types:** ``feedback`` for general comments; ``report`` for problem reporting.
- **Order linking:** Feedback can be linked to a specific order for context.
- **Status workflow:** Open → Reviewed → Resolved.

13. Order Playlists

Custom recurring order templates for repeat customers.

Endpoints

(Part of the menu app: ``/api/order-playlists/``)

Key Behaviors

- **Owner-scoped:** Each playlist belongs to a single user; users can only see/edit their own.
- **Guest-restricted:** Guest users cannot create or use playlists.
- **Bulk ordering:** A playlist can be applied to a new draft order via ``POST /api/orders/from_playlist/``, copying all items and quantities.

System Configuration

Key settings exposed in ``settings.py``:

Setting	Default	Purpose
<code>`GUEST_ACCESS_HO`</code>	12	Guest session expiry window
<code>`R_SUBMIT_DEBOUNC`</code>	5	Rate-limit window for order submission
<code>`DRY_UPDATE_DEBOU`</code>	2	Rate-limit for inventory updates
<code>`FF_PAGE_DEBOUNCE`</code>	10	Rate-limit for staff page requests
<code>`DER_PRICE_ALERT_TH`</code>	PHP 2,500.00	High-value order notification threshold
<code>`YNC_RETRY_MAX_AT`</code>	8	Max sync event delivery retries
<code>`YNC_RETRY_BASE_S`</code>	10	Exponential backoff base interval

Technology Stack

Component	Technology
Framework	Django 6.0.3 + Django REST Framework
API Documentation	drf-spectacular (Swagger UI at <code>`/api/docs/`</code>)
Authentication	Token-based (DRF TokenAuthentication)
Database	SQLite3 (development); PostgreSQL-ready via ORM
Realtime	SSE polling (1s interval, StreamingHttpResponse)
CORS	django-cors-headers (all origins allowed in dev)
ASGI	Django Channels (installed but unused; positioned for future WebSocket migration)

API Documentation

Interactive Swagger UI is available at ``/api/docs/`` and the raw OpenAPI schema at ``/api/schema/``.